쇼핑몰 기능구현

목차

1.UI

2.모델링

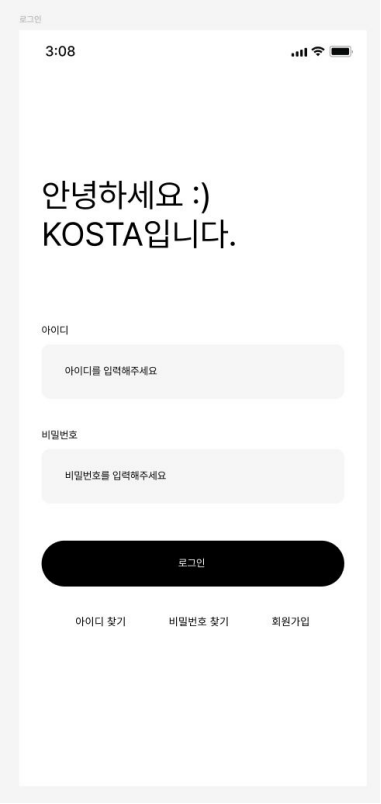
3.업무분석

4.SQL

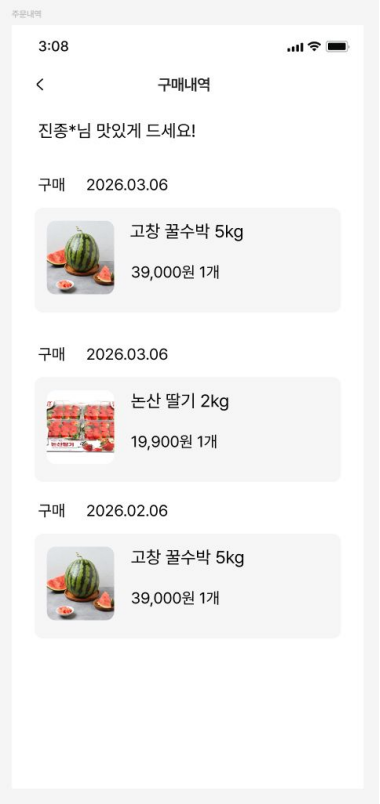
5.클래스 다이어그램

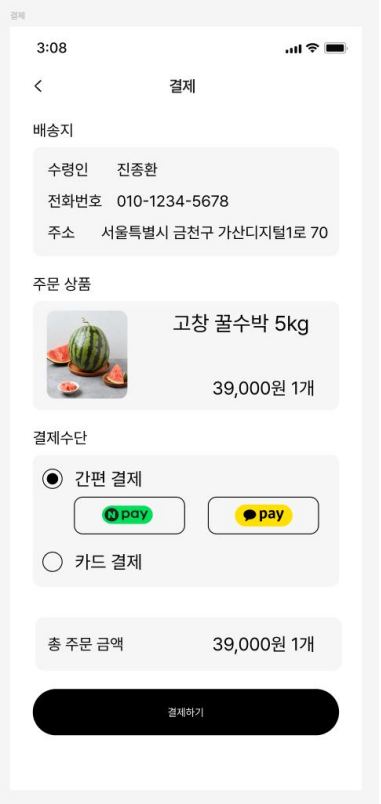
6.회고

1.UI

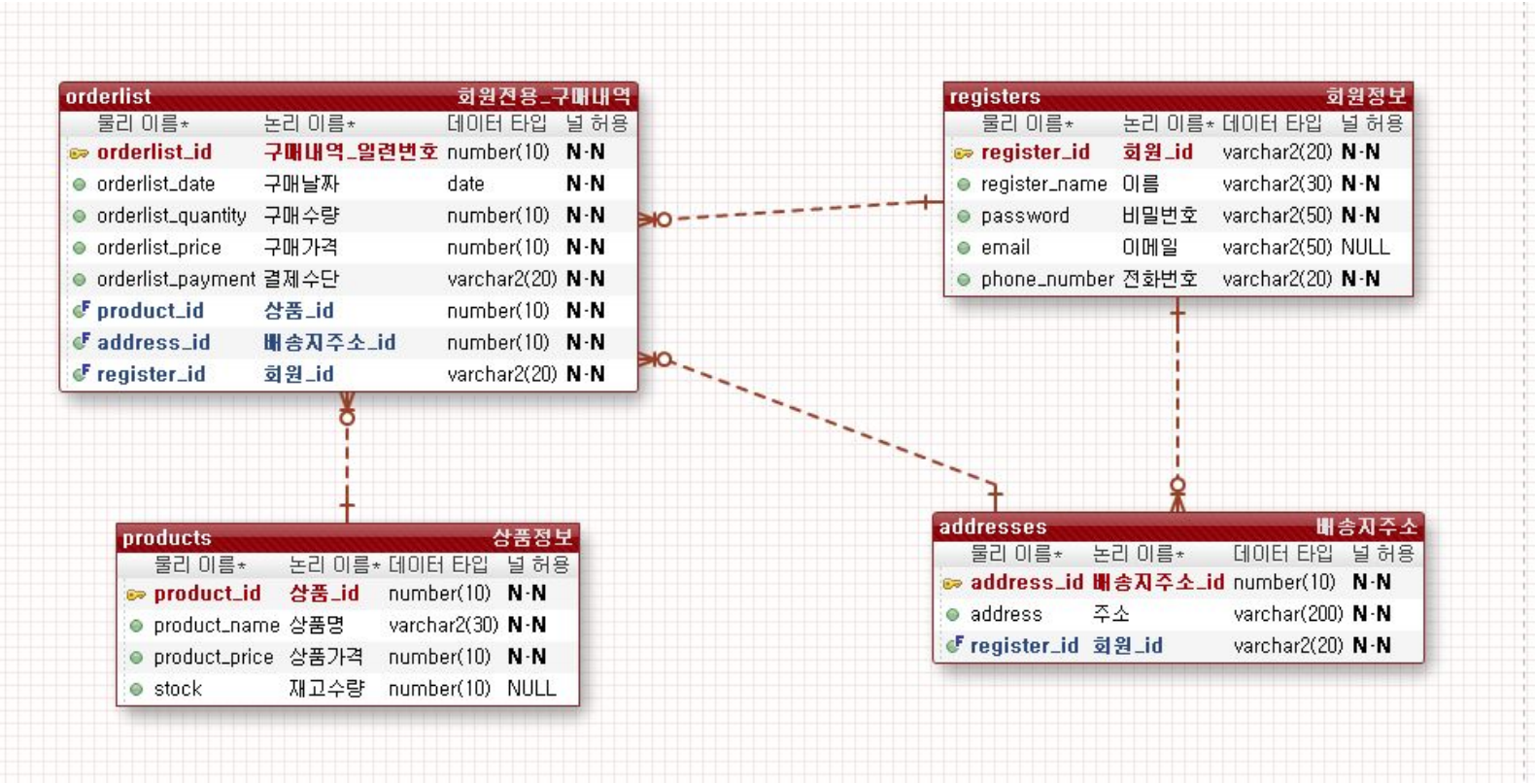

로그인
3:08
안녕하세요 :)
KOSTA입니다.
아이디
아이디를 입력해주세요
비밀번호
비밀번호를 입력해주세요
로그인
아이디 찾기
비밀번호 찾기
회원가입

회원가입
3:08
회원가입
아이디
아이디를 입력해주세요
비밀번호
비밀번호를 입력해주세요
이름
비밀번호를 입력해주세요
이메일
이메일
전화번호
전화번호
가입하기


주문내역
3:08
구매내역
진종*님 맛있게 드세요!
구매 2026.03.06
고창 꿀수박 5kg
39,000원 1개
구매 2026.03.06
논산 딸기 2kg
19,900원 1개
구매 2026.02.06
고창 꿀수박 5kg
39,000원 1개


결제
3:08
결제
배송지
수령인 진종환
전화번호 010-1234-5678
주소 서울특별시 금천구 가산디지털1로 70
주문 상품
고창 꿀수박 5kg
39,000원 1개
결제수단
간편 결제
N pay
pay
카드 결제
총 주문 금액 39,000원 1개
결제하기

2.모델링


orderlist 회원전용_구매내역
물리 이름* 논리 이름* 데이터 타입 널 허용
orderlist_id 구매내역_일련번호 number(10) N·N
orderlist_date 구매날짜 date N·N
orderlist_quantity 구매수량 number(10) N·N
orderlist_price 구매가격 number(10) N·N
orderlist_payment 결제수단 varchar2(20) N·N
product_id 상품_id number(10) N·N
address_id 배송지주소_id number(10) N·N
register_id 회원_id varchar2(20) N·N
registers 회원정보
물리 이름* 논리 이름* 데이터 타입 널 허용
register_id 회원_id varchar2(20) N·N
register_name 이름 varchar2(30) N·N
password 비밀번호 varchar2(50) N·N
email 이메일 varchar2(50) NULL
phone_number 전화번호 varchar2(20) N·N
products 상품정보
물리 이름* 논리 이름* 데이터 타입 널 허용
product_id 상품_id number(10) N·N
product_name 상품명 varchar2(30) N·N
product_price 상품가격 number(10) N·N
stock 재고수량 number(10) NULL
addresses 배송지주소
물리 이름* 논리 이름* 데이터 타입 널 허용
address_id 배송지주소_id number(10) N·N
address 주소 varchar(200) N·N
register_id 회원_id varchar2(20) N·N

3.업무분석

회원				
업무	CRUD	SQL 계열	관련 테이블	메서드
회원가입	CREATE	INSERT	registers	setRegister()
로그인	READ	SELECT	registers	login()
아이디 찾기	READ	SELECT	registers	getRegisterId()
비밀번호 변경	UPDATE	UPDATE	registers	setPassword()
상품				
업무	CRUD	SQL 계열	관련 테이블	메서드
상품 목록 조회	READ	SELECT	products	getProduct()
주문				
업무	CRUD	SQL 계열	관련 테이블	메서드
구매내역 조회	READ	SELECT	orderlist, products, registers, addresses	getOrders()
재고 확인	READ	SELECT	products	stockCheck()
상품 주문	CREATE	INSERT	orderlist	setOrder()
재고 차감 (주문 시 자동)	UPDATE	UPDATE	products	setOrder() 내부
배송				
업무	CRUD	SQL 계열	관련 테이블	메서드
배송지 등록	CREATE	INSERT	addresses	setAddress()
배송지 조회	READ	SELECT	addresses	getAddress()
배송지 수정	UPDATE	UPDATE	addresses	updateAddress()
미구현				
업무	CRUD	SQL 계열	관련 테이블	메서드
배송지 삭제	DELETE	DELETE	addresses	미구현
회원 탈퇴	DELETE	DELETE	registers	미구현
주문 취소	DELETE	DELETE	orderlist	미구현

4.SQL

1. 회원가입
INSERT INTO registers (register_id, register_name, password, email, phone_number) VALUES ('ehdehd', '최동석', '12345678', 'ehdehd@naver.com', '01088877778');

2.로그인
SELECT register_name FROM registers WHERE register_id = 'chulsoo77' AND password = 'chul1234!';

3.아이디 찾기
SELECT register_id FROM registers WHERE register_name = '김철수' AND phone_number = '010-4562-1289';

4.비밀번호 변경
UPDATE registers SET password = 'chulsoo77' WHERE register_id = 'chul1234!' AND PASSWORD = '12345678';

4.SQL

5.구매내역 조회

```
SELECT o.orderlist_id, o.orderlist_date, p.product_name, orderlist_quantity, p.product_price, orderlist_price, orderlist_payment, a.address
FROM orderlist o, products p, registers r, addresses a
WHERE r.register_id = 'chulsoo77' AND o.product_id = p.product_id AND r.register_id = o.register_id AND o.address_id = a.address_id;
```

6.주문

```
INSERT INTO orderlist (orderlist_id, orderlist_date, orderlist_quantity, orderlist_price, orderlist_payment, product_id, address_id, register_id)
VALUES (seq_orderlist_id.nextval, SYSDATE, 4, 4 * (SELECT product_price FROM products WHERE product_id = 2), '카카오페이', 2, 1, 'chulsoo77';
```

7.재고 업데이트

```
UPDATE products
SET stock = stock - 3
WHERE product_id = 1;
```

4.SQL

8.재고 체크

```
SELECT stock
FROM products
WHERE product_id = 3;
```

9.상품정보

```
SELECT product_id,
product_name, product_price, stock
FROM products;
```

10.배송지 등록

```
INSERT INTO addresses
(address_id, address, register_id)
VALUES (seq_address_id.nextval,
'인천시부평구', 'chulsoo77');
```

4.SQL

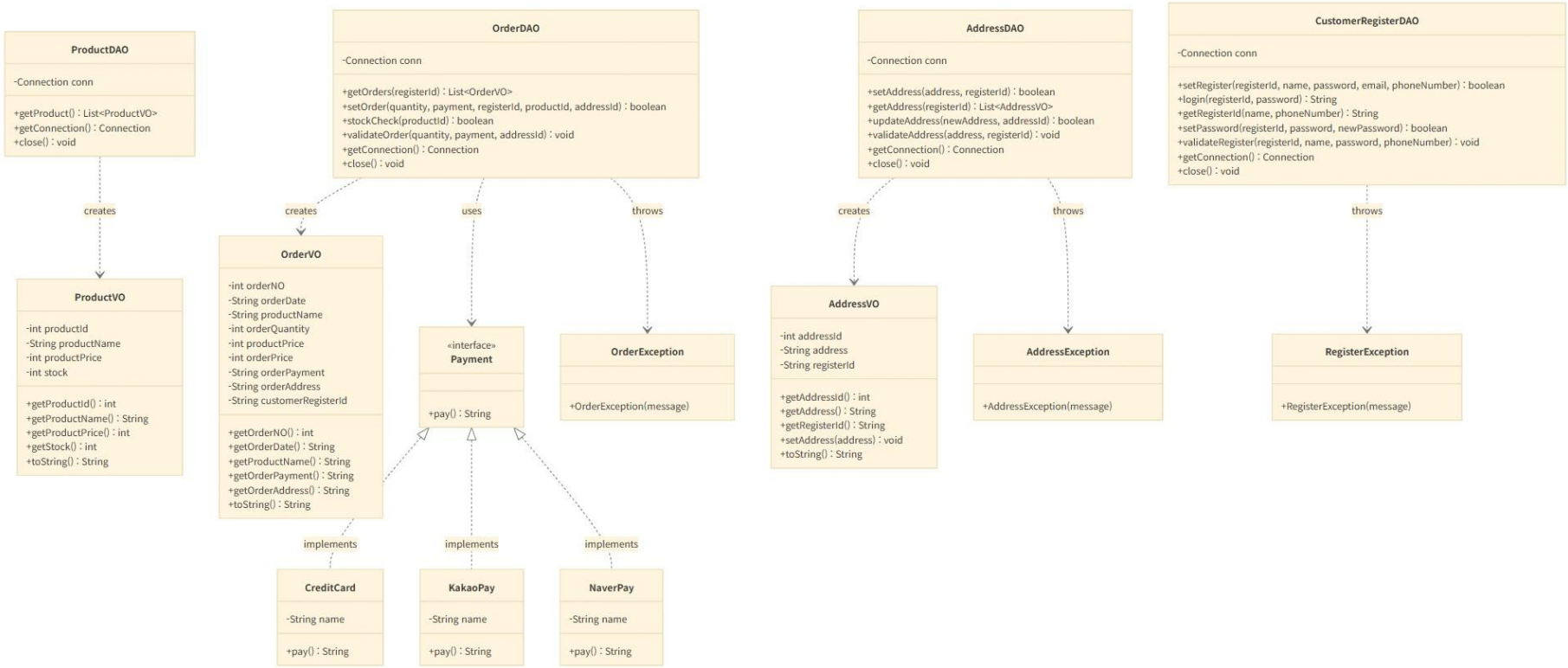
11.배송지 조회

```
SELECT address_id, address,
register_id
FROM addresses
WHERE register_id = 'chulsoo77';
```

12.배송지 수정

```
UPDATE addresses
SET address = '인천시부평구'
WHERE address_id = 'chulsoo77';
```

5.클래스 다이어그램


ProductDAO
-Connection conn
+getProduct() : List<ProductVO>
+getConnection() : Connection
+close() : void
creates
ProductVO
-int productId
-String productName
-int productPrice
-int stock
+getProductId() : int
+getProductName() : String
+getProductPrice() : int
+getStock() : int
+toString() : String
OrderDAO
-Connection conn
+getOrders(registerId) : List<OrderVO>
+setOrder(quantity, payment, registerId, productId, addressId) : boolean
+stockCheck(productId) : boolean
+validateOrder(quantity, payment, addressId) : void
+getConnection() : Connection
+close() : void
creates
uses
throws
OrderVO
-int orderNO
-String orderDate
-String productName
-int orderQuantity
-int productPrice
-int orderPrice
-String orderPayment
-String orderAddress
-String customerRegisterId
+getOrderNO() : int
+getOrderDate() : String
+getProductName() : String
+getOrderPayment() : String
+getOrderAddress() : String
+toString() : String
«interface»
Payment
+pay() : String
OrderException
+OrderException(message)
implements
implements
implements
CreditCard
-String name
+pay() : String
KakaoPay
-String name
+pay() : String
NaverPay
-String name
+pay() : String
AddressDAO
-Connection conn
+setAddress(address, registerId) : boolean
+getAddress(registerId) : List<AddressVO>
+updateAddress(newAddress, addressId) : boolean
+validateAddress(address, registerId) : void
+getConnection() : Connection
+close() : void
creates
throws
AddressVO
-int addressId
-String address
-String registerId
+getAddressId() : int
+getAddress() : String
+getRegisterId() : String
+setAddress(address) : void
+toString() : String
AddressException
+AddressException(message)
CustomerRegisterDAO
-Connection conn
+setRegister(registerId, name, password, email, phoneNumber) : boolean
+login(registerId, password) : String
+getRegisterId(name, phoneNumber) : String
+setPassword(registerId, password, newPassword) : boolean
+validateRegister(registerId, name, password, phoneNumber) : void
+getConnection() : Connection
+close() : void
throws
RegisterException
+RegisterException(message)

6.회고

- 최동석

이 프로젝트를 하면서 가장 많이 느낀 것은 모델링이 끝났으면 건들지 않는다는 것입니다.

모델링이 끝나고 그에 맞게 구현할 때 모델링의 몇몇 부분이 마음에 들지 않을 때가 있었는데, 그럴 때마다 하나씩 바꾸다 보면 속성 하나를 추가하거나 없애는 것이 구현 코드에서는 그보다 몇 배는 더 많은 수정이나 삭제로 이어진다는 것을 알았습니다. 강사님이 말씀하신 것을 몸소 체험할 수 있어서 좋았습니다.

아쉬운 점은 회원탈퇴나 주문취소 등 구현하지 못한 기능들이 있다는 것이고, 결제 부분도 카카오페이, 네이버페이처럼 단순히 문자열을 반환하는 방식이라 아쉬움이 남았습니다. 다음에는 실제 API 연동을 통해 구현해보고 싶습니다.

그래도 ERD 설계부터 실제 구현까지 전체적인 흐름을 경험한 것과 인터페이스로 결제 수단 확장 구조를 만들어본 것은 기억에 오래 남을 것 같습니다. 다음 프로젝트에서는 모델링 단계에서 요구사항을 더욱 꼼꼼히 검토해야겠다고 느꼈습니다.